In this way a single object permeates throughout the entire system and can be used at any point with confidence that it's always the same object acted upon. This is a pattern known as the *Singleton*. It has obvious advantages for objects that are perhaps expensive to create, or hold a lot of data, or never need to be duplicated. It forms the basis of other patterns and we'll be looking at some of these neat tricks when we explore *design patterns* in Chapter 10, "Programming with Objects."

Interfaces

The public members of a class form its interface. These publicly accessible properties and methods are how an object of that class is intended to be used. When writing documentation for a class, it is normal convention to give only the names and types of public properties, public constants, and the signatures (names, parameters, return types) of public methods.

Here's a sample from the official PHP documentation for the `SimpleXMLElement` object interface:

```
class SimpleXMLElement implements Stringable, Countable,
RecursiveIterator {
/* Methods */
public __construct(
    string $data,
    int $options = 0,
    bool $dataIsURL = false,
    string $namespaceOrPrefix = "",
    bool $isPrefix = false
)
public addAttribute(string $qualifiedName, string $value, ?string
$namespace = null): void
public addChild(string $qualifiedName, ?string $value = null, ?string
$namespace = null): ?SimpleXMLElement
public asXML(?string $filename = null): string|bool
//and so on and so on
```